

EXPERIMENT-21

1. Problem Analysis:

Input:

Two positive integers m and n, where $m \leq n$

Menu choice from 1 to 5

Operations:

1. Print first 10 multiples of every integer from m to n.
2. Print addition tables using addends 1 to 10.
3. Print subtraction tables using subtrahends 1 to 10.
4. Divide every integer by 1 to 10 and display results to two decimal places.
5. Exit.

Concepts Used:

do-while loop for repeatedly displaying the menu.

switch-case for menu selection.

Nested for loops for generating tables.

Input validation for m, n, and menu choice.

Type casting to double for division.

2. Algorithm:

Algorithm: Menu-Driven Tables

1. Start.
2. Declare integer variables m, n, choice, i, and j.
3. Input m and n.

4. Check whether:

$m > 0$

$n > 0$

$m \leq n$

5. If the input is invalid:

Display an appropriate error message.

Ask the user to enter m and n again.

6. Repeat Steps 3–5 until valid values are entered.

7. Start a do-while loop.

8. Display the menu:

1. Multiplication Tables

2. Addition Tables

3. Subtraction Tables

4. Division Tables

5. Exit

9. Input choice.

10. Use a switch statement.

11. If choice = 1:

Repeat i from m to n.

For each i, repeat j from 1 to 10.

Display $i \times j = i*j$.

12. If choice = 2:

Repeat i from m to n.

For each i, repeat j from 1 to 10.

Display $i + j = i+j$.

13. If choice = 3:

Repeat i from m to n.
 For each i, repeat j from 1 to 10.
 Display $i - j = i - j$.

14. If choice = 4:
 Repeat i from m to n.
 For each i, repeat j from 1 to 10.
 Calculate i / j using floating-point division.
 Display the answer rounded to two decimal places.

15. If choice = 5:
 Display an exit message

16. For any other choice:
 Display "Invalid menu choice."

17. Repeat the menu until choice = 5.

18. Stop.

3. Test-Case Table:

The table includes valid, invalid, and boundary cases.

TC	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	m=2, n=4, Choice 1	Multiplication tables of 2, 3 and 4 from $\times 1$ to $\times 10$	Same as expected	Pass
2	m=2, n=3, Choice 2	Addition tables of 2 and 3 using 1–10	Same as expected	Pass
3	m=5, n=6, Choice 3	Subtraction tables of 5 and 6 using 1–10	Same as expected	Pass
4	m=2, n=3, Choice 4	Division tables of 2 and 3 with results to 2 decimal places	Same as expected	Pass
5	m=1, n=1, Choice 1	Multiplication table of 1	Same as expected	Pass
6	m=5, n=5, Choice 4	Division table of 5	Same as expected	Pass
7	m=0, n=5	Error; request values again	Error displayed	Pass
8	m=-2, n=5	Error; request values again	Error displayed	Pass
9	m=5, n=3	Error because $m > n$; request again	Error displayed	Pass
10	m=2, n=5, Choice 0	Invalid menu choice	Error displayed	Pass
11	m=2, n=5, Choice 6	Invalid menu choice	Error displayed	Pass
12	m=2, n=5, Choice 5	Program exits	Program exits	Pass

Boundary Cases:

$m = n$ is valid because the condition is $m \leq n$. In this situation, the program simply prints the selected table for one integer.

4. Program, Compile and Execution:

```
GNU nano 9.0
#include <stdio.h>

int main()
{
    int m, n, choice, i, j;

    /* Input and validation of m and n */
    do
    {
        printf("Enter two positive integers m and n ( $m \leq n$ ): ");
        scanf("%d %d", &m, &n);

        if ( $m \leq 0$  ||  $n \leq 0$ )
        {
            printf("Error: m and n must be positive integers.\n");
        }
        else if ( $m > n$ )
        {
            printf("Error: m must be less than or equal to n.\n");
        }
    } while ( $m \leq 0$  ||  $n \leq 0$  ||  $m > n$ );

    /* Menu-driven program */
    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Print Multiplication Tables\n");
        printf("2. Print Addition Tables\n");
        printf("3. Print Subtraction Tables\n");
        printf("4. Print Division Tables\n");
        printf("5. Exit\n");
        printf("=====\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);
    }
```

GNU nano 9.0

```
switch (choice)
{
    case 1:
        printf("\n— MULTIPLICATION TABLES —\n");

        for (i = m; i ≤ n; i++)
        {
            printf("\nTable for %d:\n", i);

            for (j = 1; j ≤ 10; j++)
            {
                printf("%d x %d = %d\n",
                    i, j, i * j);
            }
        }
        break;

    case 2:
        printf("\n— ADDITION TABLES —\n");

        for (i = m; i ≤ n; i++)
        {
            printf("\nAddition table for %d:\n", i);

            for (j = 1; j ≤ 10; j++)
            {
                printf("%d + %d = %d\n",
                    i, j, i + j);
            }
        }
        break;

    case 3:
        printf("\n— SUBTRACTION TABLES —\n");

        for (i = m; i ≤ n; i++)
        {
            printf("\nSubtraction table for %d:\n", i);

            for (j = 1; j ≤ 10; j++)
            {
                printf("%d - %d = %d\n",
                    i, j, i - j);
            }
        }
        break;
}
```

```

    case 4:
        printf("\n— DIVISION TABLES —\n");

        for (i = m; i ≤ n; i++)
        {
            printf("\nDivision table for %d:\n", i);

            for (j = 1; j ≤ 10; j++)
            {
                printf("%d / %d = %.2f\n",
                    i, j, (double)i / j);
            }
        }
        break;

    case 5:
        printf("\nExiting the program.\n");
        break;

    default:
        printf("\nError: Invalid menu choice. "
            "Please enter a choice from 1 to 5.\n");
}

} while (choice ≠ 5);

return 0;
}

```

```
(kali㉿kali)-[~]  
$ nano 21.c
```

```
(kali㉿kali)-[~]  
$ gcc 21.c -o 21
```

```
(kali㉿kali)-[~]  
$ ./21
```

Enter two positive integers m and n ($m \leq n$): 2
3

```
===== MENU =====  
1. Print Multiplication Tables  
2. Print Addition Tables  
3. Print Subtraction Tables  
4. Print Division Tables  
5. Exit
```

Enter your choice: 1

— MULTIPLICATION TABLES —

Table for 2:

2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20

Table for 3:

3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30

```
===== MENU =====  
1. Print Multiplication Tables  
2. Print Addition Tables  
3. Print Subtraction Tables  
4. Print Division Tables  
5. Exit
```

```
=====
```

Enter your choice: 2

```
—— ADDITION TABLES ——
```

Addition table for 2:

```
2 + 1 = 3  
2 + 2 = 4  
2 + 3 = 5  
2 + 4 = 6  
2 + 5 = 7  
2 + 6 = 8  
2 + 7 = 9  
2 + 8 = 10  
2 + 9 = 11  
2 + 10 = 12
```

Addition table for 3:

```
3 + 1 = 4  
3 + 2 = 5  
3 + 3 = 6  
3 + 4 = 7  
3 + 5 = 8  
3 + 6 = 9  
3 + 7 = 10  
3 + 8 = 11  
3 + 9 = 12  
3 + 10 = 13
```

```
===== MENU =====
1. Print Multiplication Tables
2. Print Addition Tables
3. Print Subtraction Tables
4. Print Division Tables
5. Exit
```

```
Enter your choice: 3
```

```
File system
— SUBTRACTION TABLES —
```

```
Subtraction table for 2:
```

```
2 - 1 = 1
2 - 2 = 0
2 - 3 = -1
2 - 4 = -2
2 - 5 = -3
2 - 6 = -4
2 - 7 = -5
2 - 8 = -6
2 - 9 = -7
2 - 10 = -8
```

```
Subtraction table for 3:
```

```
3 - 1 = 2
3 - 2 = 1
3 - 3 = 0
3 - 4 = -1
3 - 5 = -2
3 - 6 = -3
3 - 7 = -4
3 - 8 = -5
3 - 9 = -6
3 - 10 = -7
```

```
===== MENU =====
1. Print Multiplication Tables
2. Print Addition Tables
3. Print Subtraction Tables
4. Print Division Tables
5. Exit
=====
```

Enter your choice: 4

— DIVISION TABLES —

Division table for 2:

```
2 / 1 = 2.00
2 / 2 = 1.00
2 / 3 = 0.67
2 / 4 = 0.50
2 / 5 = 0.40
2 / 6 = 0.33
2 / 7 = 0.29
2 / 8 = 0.25
2 / 9 = 0.22
2 / 10 = 0.20
```

Division table for 3:

```
3 / 1 = 3.00
3 / 2 = 1.50
3 / 3 = 1.00
3 / 4 = 0.75
3 / 5 = 0.60
3 / 6 = 0.50
3 / 7 = 0.43
3 / 8 = 0.38
3 / 9 = 0.33
3 / 10 = 0.30
```

```
===== MENU =====
1. Print Multiplication Tables
2. Print Addition Tables
3. Print Subtraction Tables
4. Print Division Tables
5. Exit
=====
```

Enter your choice: █

11. Invalid Input Testing:

Invalid m and n:

Enter two positive integers m and n ($m \leq n$): 0 5

Error: m and n must be positive integers.

Enter two positive integers m and n ($m \leq n$): 8 3

Error: m must be less than or equal to n.

Enter two positive integers m and n ($m \leq n$): 2 5

The program accepts the final input and continues.

Invalid Menu Choice:

Enter your choice: 7

Error: Invalid menu choice. Please enter a choice from 1 to 5.

The program then displays the menu again rather than terminating.

Exit

This satisfies the requirement that the menu keeps repeating until the user explicitly selects Exit.

NAME: TAANISH AGARWAL

BATCH: B-15

SAP ID: 590034662